ChatGPT

Upgrading Commodore 64/128 as Secure Robotics Terminals (2025)

Overview: *Vintage Commodore 8-bit machines can be remarkably adapted into secure access terminals for modern systems. With today's hardware expansions and clever interfacing, a C64 or C128 can bridge 1980s architecture with 2025 robotics. Below we explore performance upgrades, architectural mods, I/O integration, software stacks, and security setups – all geared for expert users aiming to repurpose these classics as reliable, hardened terminals in a robotics project (e.g. the Monkey-Head-Project).*

Performance Enhancements in 2025

Modern CPU Accelerators: Despite the C64's stock 1 MHz 6510 CPU, several contemporary accelerators dramatically boost speed. The new **TurboMaster V3** (2025 re-release) swaps in a 4 MHz 65C02 CPU (4× original speed) with fast SRAM and built-in JiffyDOS [1] [2]. It's compatible with C64 (and C128 in C64-mode), featuring toggle switches for 1 MHz/4 MHz operation and ROM selection [3]. Enthusiasts can also seek the legendary **Commodore SuperCPU** (20 MHz 65816 CPU with up to 16 MB RAM) – though long discontinued, it set the bar for C64 acceleration. Newer projects even exceed SuperCPU performance: the **MCL64** accelerator uses a Teensy 4.1 microcontroller to emulate a 6510 CPU at effective clock rates >600 MHz [4]. In testing, this open-source board ran 2× faster than a SuperCPU at 20 MHz [5], yet costs under $50 [6]. For the C128, built-in dual CPUs (2 MHz 8502 and Z80) already permit modest speed-ups – the 128 can double its 8502 clock when the 40-column VIC display is off (e.g. in 80-col mode), improving performance for purely computational tasks.

Memory Expansions: Both C64 and C128 benefit from greatly expanded RAM. Commodore's vintage RAM Expansion Units (**REU**) – originally 128 KB to 512 KB – have modern equivalents offering **16 MB** or more. For example, the **RAD Expansion Unit** is a recent Raspberry Pi-powered cartridge that provides up to 16 MB RAM as an REU-compatible memory pool [7]. The **TurboMaster's Master Adapter** (revived alongside V3) also supports REUs or GEOS-specific RAM expansions [8]. These memory boosts enable larger terminal buffers, RAM disks, or even multitasking OS environments on the Commodore. Other multi-function carts like the **Turbo Chameleon 64** include 16 MB REU emulation and 4 MB GeoRAM support [9]. In practice, a RAM expansion allows running more complex client software (for instance, a GUI OS or networking stack) and can cache data from the host system for smoother interaction.

Modern Multi-Function Cartridges: FPGA-based cartridges provide all-in-one performance and I/O enhancements. The **Chameleon 64** (by Individual Computers) exemplifies this: it docks in the C64's expansion slot and adds *a VGA video output, SD card storage, up to 16 MB RAM, freezer/debugger, and a "turbo mode" CPU core*, all while remaining compatible with original software [10] [11]. An optional RR-Net module even gives Ethernet networking [10]. Such devices can accelerate the Commodore (e.g. 3–4× normal speed), reduce load times, and interface with modern displays and networks seamlessly – hugely beneficial for a terminal unit.

Storage and Peripherals: While disk drives are less relevant for a terminal, fast solid-state storage helps in loading software and logs. SD-card based drive emulators (e.g. SD2IEC or 1541 Ultimate II+) and flash cartridges (e.g. EasyFlash 3) provide quick program access. EasyFlash 3 can also hold multiple KERNAL ROMs (even booting **JiffyDOS** or custom terminal ROMs without hardware mod) [12], which is convenient for expert setups. Additionally, C128 users might leverage 1571/1581 disk drives or IEEE-488 interfaces for larger storage, though network-based transfer often supersedes physical media in 2025.

Architecture Modifications & Modern Co-Processors

Microcontroller & FPGA Integration: A popular approach is augmenting the Commodore with a co-processor that handles heavy tasks while the 8-bit machine remains the user interface. The **Idun-Cartridge** (2025) is a prime example: a custom cart for C128/C64 that embeds a Parallax Propeller microcontroller interfacing the Commodore's bus to a Raspberry Pi Zero 2 running Linux [13] [14]. The Propeller handles real-time bus signals, effectively acting as an MMU/bridge, while the Pi provides high-level processing and networking. This architecture gives the Commodore an *MS-DOS-like shell with instant app launching*, "ERAM" memory (4 MB on Idun cart), and a built-in VT100 terminal to the Pi's Linux system [15] [16]. In practice, the Commodore drives a Pi console securely – you can Telnet/SSH into modern services via the Pi, use the Pi's Wi-Fi/Ethernet for connectivity, and even run 6502 or Z80 code that interacts with the Pi through an API [17] [18]. This essentially turns the C64/128 into a smart terminal with a Linux backend. (Notably, Idun also supports *Lua scripting and assembly coding* against its API, letting developers create custom terminal applications that exploit the Pi's resources [17].) Another community project, the **Sidekick64**, similarly uses a Raspberry Pi in a cart to emulate various hardware (REU, drives, etc.) and can run PETSCII graphical launchers – again marrying modern compute with the C64's IO. These solutions illustrate how *"plugging a contemporary computer INTO your C64"* yields extensive new capabilities [19] while preserving the retro front-end.

Internal CPU Replacements: Short of adding external CPUs, one can also replace or augment the Commodore's CPU internally. The MCL64's Teensy-based 6510 replacement (mentioned above) is one such mod, directly swapping the CPU for a much faster emulated core [6]. There are also prototype projects using 65816 or WDC 65C02 CPUs as drop-in upgrades for C64/C128, offering higher clocks and CMOS improvements. The C128's architecture is inherently more complex – it has an MMU and a Z80 for CP/M – but some accelerators (like TurboMaster or SuperCPU) can operate in C128's 64-mode, and hypothetically a custom accelerator could target 2 MHz 128-mode as well [20]. Additionally, creative hackers have doubled the C128's Z80 clock in experiments [21], which could expedite CP/M software (if, for example, using CP/M terminal programs). While such CPU mods require deep hardware knowledge, they demonstrate the potential to **tune the Commodore's architecture for better integration** with modern systems (faster data encryption, snappier terminal responses, etc.).

Firmware and OS Mods: Upgrading the KERNAL ROM or using alternative OS can facilitate modern integration. Installing **JiffyDOS** (a fast disk/serial DOS ROM) accelerates disk operations and provides convenient BASIC commands – useful if the terminal needs to load/configure files quickly. It also speeds up serial device transfers (helpful when using user-port devices). On the C128, enabling **CP/M mode** and running CP/M software (like Kermit or IMP terminal programs) is another tactic – the Z80 CP/M environment might interface with vintage mainframes or CP/M-speaking devices. However, most robotics projects will prefer native mode for full graphics/sound if needed. Some hobby OSes like **GEOS** (and its WiNGs/Wheels enhancements) can run on expanded RAM and even support rudimentary TCP/IP or GUI terminals, but

generally a lightweight approach (bare-metal terminal software or Contiki, see below) is favored for security and simplicity.

I/O Integration Methods (User Port, Serial & Expansion)

Adapting a Commodore as a robotics terminal hinges on robust I/O connectivity. Fortunately, the C64/C128 offer multiple interfacing ports: the **User Port**, the **Serial/IEC bus**, and the **Cartridge Expansion slot** – each with its own use cases.

- **User Port (TTL Serial & GPIO):** The user port is a 24-pin edge connector exposing one 8-bit parallel I/O port (CIA2 Port B bits PB0–PB7) plus a few control lines (FLAG2, PC2, etc.) and serial lines from the CIAs [22] [23]. It's very versatile – essentially a GPIO breakout – and commonly used for serial communications. By adding a level shifter (e.g. MAX232) one can achieve RS-232 voltage levels and connect the user port to modern serial devices or microcontrollers. For instance, a simple circuit with a MAX232 and inverters can convert the C64's TTL TX/RX lines to a DB9 serial port supporting up to **9600 bps (UP9600 mode)** [24] [25]. Many hobbyists have built such adapters to link the C64 to PCs or to vintage modems. Today, this is used with **ESP8266/ESP32 WiFi modem boards** that plug into the user port, letting the C64 "dial out" to Internet hosts via telnet. The user port's serial lines (CNT, SP pins) can achieve around 2400 bps using the KERNAL routines, but with custom routines or hardware assist (UP9600 hack or SwiftLink cartridge) speeds of 9600–38400 bps are attainable [26]. Notably, **connecting to microcontrollers** is straightforward: in one project, a C64 was linked to an Arduino at 2400 bps over the user port to serve as a graphical front-end for sensor and actuator data [27]. The Commodore ran a Firmata client that could toggle Arduino outputs and read inputs, using the C64's screen to display status of LEDs, relays, servos, etc. [27] [28]. This demonstrates using the user port as a **"GPIO extender"** – the 8 data lines can be manipulated in BASIC or assembly (POKEing the DDR register and data register at $DD03/$DD01) to bit-bang signals out to custom hardware [29]. For robotics, one could directly drive simple devices (LED indicators, relays) or read digital sensors through these lines. However, for complex interactions or higher bandwidth, a serial or cartridge link is preferred. *(Tip: when wiring the user port, include level translation and perhaps protection – one hacker suggests optocouplers for safeguarding the CIA chip from voltage spikes in experimental setups [30] [31]. At minimum, ensure common ground and never exceed 5V on any pin.)* The user port also provides two 5V power pins and even 9VAC, which can power small external circuits within its current limits [22] – convenient for modest add-on boards.

- **Expansion/Cartridge Port:** This is the Commodore's primary expansion bus, exposing address/data lines, CPU control signals, power, and more. Cartridges here can interface at a very low level – effectively becoming part of the system bus. For modern upgrades, the expansion slot is used by accelerators, memory expansions, network cards, etc. For example, the **RR-Net** Ethernet cartridge (often paired with the Retro Replay or Chameleon) sits on the expansion bus and maps a simple Ethernet controller into memory, allowing IP networking on C64. In a robotics context, an expansion port solution could be a custom cartridge containing a microcontroller or FPGA that communicates with the Commodore *as memory-mapped I/O*. This yields high throughput and precise timing (since the cart can respond to reads/writes each CPU cycle). The Idun and Sidekick64 cartridges mentioned earlier take this approach by inserting a Propeller or Pi to handle events. Another novel cartridge, the **RAD** FPGA/Pi expansion, goes so far as to render full-motion video (even *running DOOM* on a Pi and streaming each frame to the C64 screen via the cartridge) [32] – effectively turning the C64 into a display terminal for Pi-generated content. For less extreme uses, one might design a cartridge that

interfaces to an SPI/I²C bus, allowing the C64 to talk to modern sensors or microcontrollers at high speed. In fact, commentors have mused that a "32-bit co-processor in the cart slot" could serve as a crypto engine or AI co-processor, feeding data back to the C64 in manageable chunks [33] [34]. The expansion port also allows **multi-port interfaces**: e.g. the Chameleon and Ultimate64 provide PS/2 keyboard/mouse inputs, VGA video, audio output, etc., by tapping into the C64 bus and substituting or duplicating signals [10]. For a secure terminal, one could plug in a cartridge that gives a modern video output (VGA/HDMI) – helpful since finding reliable composite or RF monitors is difficult now. (The Chameleon's VGA output mirrors the VIC-II output digitally, producing a crisp 31 kHz image [35] [36].) In summary, the expansion slot is the gateway for deep hardware mods – whether it's adding an Ethernet NIC, a GPU, extra serial ports (e.g. the classic SwiftLink or Turbo232 RS232 carts), or a full-blown secondary computer.

- **IEC Serial (CBM Bus):** The Commodore's disk/printer serial bus (the round DIN connectors) is another integration path, though more specialized. It's essentially a slow (~1 kB/s) serial daisy-chain network used for 1541 drives, etc., not to be confused with RS-232. Some hackers have co-opted the IEC bus to connect microcontrollers that **emulate disk drives or other peripherals**. For instance, projects like **sd2iec** and **Pi1541** use microcontrollers/Pi to act as a 1541 drive on the IEC bus, allowing the C64 to load data from SD cards or even receive commands. In a robotics project, one could theoretically present the robot controller as a "disk drive" device to the Commodore – the C64 could send commands by opening files or writing to the drive, and the controller could respond as if delivering disk data. This is clever but not the most real-time interface (and would require custom DOS commands or a protocol). A more direct approach is using the user port or cartridge as described above. Still, it's worth noting that modern microcontroller libraries exist for the IEC bus (e.g. Arduino IEC libraries) [37], meaning an Arduino/Pi could interface on that bus if needed (perhaps as a secondary channel for logging or failsafe communication with the C64).

- **Miscellaneous I/O:** The C64/C128 also have joystick ports (basically 5 digital inputs + 2 analog paddles each) which can be repurposed to connect simple binary sensors or trigger inputs (for example, a panic stop button for the robot could be wired to a joystick port pin and read via the CIA register $DC01). The C128 in particular has an **RGBI video output** for 80-column mode; while not an input port, leveraging the 80-col output can be important for a terminal (discussed below in software). If using a C128, one might invest in an RGBI→VGA or RGBI→SCART converter to use a modern monitor for the 80-column screen [38] [39]. Additionally, the C128 has a user port and cassette port much like the C64 (the cassette port can be abused for digital I/O or power in some cases, but it's rarely needed when user port is available).

Integration Example – Pi Zero via User Port: To illustrate a real-world wiring: one project attached a Raspberry Pi Zero W directly to the C64 user port to grant it Linux capabilities. The Pi's TX/RX GPIO pins were level-shifted (via a 74HC245 buffer) to the C64's user port lines, and the Pi's composite video out was connected to a Commodore monitor input [40] [41]. This setup allowed a standard C64 terminal program to log into the Pi's console (the Pi ran a serial console login on its UART), effectively giving a Linux shell on the C64 screen [42]. Both the C64 and Pi ran concurrently, meaning the C64 could also display its own screen or combine outputs – one could even toggle between the C64's own display and the Pi's video signal on a monitor. The creator noted this opens up possibilities like running **TCP/IP services (SSH, etc.)** on the Pi and using the C64 as a terminal to those – something beyond the reach of simpler "WiFi modem" devices [43]. Indeed, typical user-port WiFi modems only handle unencrypted telnet (PETSCII or ASCII), but with a Pi involved, you can achieve a **secure SSH connection** from the C64 to a modern host (the Pi does the

encryption, the C64 just sees the plaintext over the local serial link). This is a powerful integration for security-minded applications, allowing the vintage terminal to interface with secure networks.

In summary, **user port** is great for UART/GPIO links to microcontrollers or single-board computers (simple wiring, moderate speeds), **expansion port** is ideal for high-speed and feature-rich interfaces (via cartridges or custom hardware), and the **IEC/other ports** are niche options for specific compatibility needs. By selecting the right interface, you can connect a Commodore terminal to nearly anything – from directly toggling robot controls to serving as a human-readable console for an advanced AI computer.

Software Stacks & Protocols for Modern Terminals

Upgrading hardware is half the battle – equally important is software that enables the Commodore to communicate and interact with modern systems. Fortunately, the retro community has produced **modernized software** that runs on C64/128, providing terminal emulation, networking protocols, and even multitasking OS features. Below are key components of a viable software stack:

- **Terminal Emulation Programs:** These are the heart of using a Commodore as an access terminal. Classic programs like *CCGMS, Novaterm,* and *Desterm* have seen updates to support contemporary needs. **CCGMS Future** (updated through 2022) is a popular C64 terminal program now featuring both **40-column and 80-column** text modes, PETSCII/ASCII↔ANSI translation, and support for higher serial baud rates and devices [44]. For example, CCGMS Future can use the user port at 2400 bps or "UP9600" fast mode up to 9600 bps, and it supports **SwiftLink cartridges** at addresses $DE00/$DF00 (6551 UART) for speeds up to 38,400 bps [45] [46]. It even includes X/Y/Zmodem file transfer protocols and an address book for dialing IP addresses or BBS numbers [46] [47]. On the C128, an **80-column terminal** is highly desirable for readability and compatibility with modern systems. Here, **DesTerm128** is an excellent choice – it runs in the C128's native 80×25 screen and supports ANSI color graphics, making connecting to Unix/Linux systems or BBSes much more user-friendly [48] [49]. A user recently documented using DesTerm128 to log into an **ANSI-based BBS** and praised it for full ANSI and 80-column support [50]. They were able to use WiFi modem hardware to connect over telnet, once configured properly. DesTerm (and some CCGMS variants) allows selecting different character encoding (PETSCII vs ASCII) and handle modern ANSI color codes, so your Commodore can display colored text UIs or menus from a Linux host or BBS. **KipperTerm 2** is another modern terminal for C64 that is oriented toward internet use – originally designed to work with Ethernet cartridges (RR-Net) or WiFi modems to do telnet, IRC, and even simple web navigation on C64. It doesn't require an OS – just run the PRG, configure the serial or TCP device, and it puts you at a ready prompt to connect to IP or serial endpoints.

- **Network Stacks & OS:** If one intends to connect the Commodore directly to TCP/IP networks (without an intermediate Pi), there are small-footprint IP stacks available. **Contiki OS** is famous for bringing multitasking and networking to 8-bitters. *Remarkably, Contiki on the C64 supports preemptive multitasking and includes a full TCP/IP stack (uIP)* with apps like a web server, web browser, Telnet, FTP and IRC clients [51] [52] – all in around 64 KB of RAM! This was demonstrated as early as 2004; today's expanded memory could allow even more. Contiki can run via an Ethernet cartridge or even slip (serial PPP) to a host. In a robotics use-case, one could run a Contiki web server on the C64 to serve a status page, or use its Telnet client to log into a control system. However, Contiki's usefulness is limited by the C64's speed – for anything cryptographic or heavy, you'd lean on a co-processor (or simply use the Commodore as a front-end to a Pi). Still, it's a proof of concept that **full IP**

networking is possible on a stock Commodore [53] [51]. Other OS options include **LUnix (Little Unix)**, a UNIX-like shell for C64 with TCP/IP support, and **GEOS** (a GUI OS for C64/128) which in the Wheels distribution gained limited TCP connectivity. Those are more for enthusiasts; a lean terminal program is usually preferable for reliability. If using a C128, CP/M mode allows running CP/M terminal software (like *Kermit-80* or *IMP*) which could be useful if interfacing with vintage mainframes or CP/M-based systems. But for modern systems, the native mode with ANSI terminal is better.

- **Protocol Bridges (TCPser, etc.):** In practice, many Commodore "networking" setups rely on a PC-side helper program such as **TCPSer** (by Jim Brain) [50]. TCPSer acts as a virtual modem: the C64 thinks it's dialing out to a modem over serial, but TCPSer (running on an attached PC or Pi) intercepts those calls and routes them over Telnet/SSH. This is how a C64 can connect to internet BBSes or servers without itself speaking TCP – the PC/Pi does the heavy lifting. For example, you might run TCPSer on a Raspberry Pi that is connected via user port or SwiftLink to the Commodore; then when you ATDT to an IP, TCPSer opens that network connection. Many retro network users employ this, as it preserves the authentic software on the C64 (just using modem commands). In a robotics project, you could similarly have the Commodore issue command strings that the Pi (via a small daemon) translates into actions or network requests. **WiFi modems** (ESP8266-based) essentially embed a mini TCPSer in firmware – you "dial" an IP:port and the device connects your C64's serial to that socket. They usually support plaintext telnet and some basic SSL (rarely full TLS). If encryption is needed, as noted, a general-purpose SBC like Pi handling SSH is better.

- **Customized Interfaces:** Depending on your robotics control software, you might write a custom client on the Commodore. For instance, if the robot exposes a simple text menu over serial, a BASIC or assembly program on the C64 could automatically boot up and interact with that menu (with special screen handling, etc.). Some hobbyists have even written **PETSCII-based GUI frameworks** – leveraging the Commodore's block graphics to create rudimentary GUI elements (windows, buttons) for controlling devices. If your use-case needs more than a text console (say, a real-time sensor dashboard), you could harness the C64's graphics modes. One could envision a program that receives telemetry from the robot (via user port or network) and plots it using VIC-II graphics – for example, drawing gauges or plotting points. This requires programming the Commodore directly, but it's feasible given its capability. In 2019, a project connected an Arduino to a C64 and displayed live sensor readings in a custom C64 program, effectively turning the C64 into a retro-style control panel [27]. The **Charm** of using the C64 here is not just nostalgia – its **"charming retro graphics"** can be an advantage for quick-glance status displays [28] (big PETSCII characters, etc.), and it avoids complex GUIs that might hide important info.

- **80-Column vs 40-Column:** It's worth emphasizing the benefit of the C128's 80-column mode for software. Many modern systems (Linux, etc.) assume at least 80-column terminals (like VT100). Using a C128 with DesTerm128 or similar in 80×25 mode allows the full breadth of text to be seen without awkward line wraps. One blogger chronicling their C128 BBS adventures noted how satisfying it was to finally see **ANSI BBS art in proper 80-column format** on a real 1084 monitor [54] [55]. For a robotics terminal, 80-cols means easier reading of logs and longer command lines. If using a C64 (40-col only), some software (CCGMS Future, Novaterm 9.6) implements an *80-column software mode* by using bitmap graphics to show 80-column text. This exists (e.g. **80x25 mode** by Michael Steil [56]), but it's slow and low-contrast on a C64. So if available, the C128's VDC output is gold – just be sure to get an RGBI-to-VGA converter or a monitor that can handle it. Alternatively, one can use a cart like Chameleon that provides VGA output with a doubled pixel clock; it doesn't increase text columns

beyond 40 (except in its own core), but at least the output is crisp. There are also mods to upgrade the VDC chip RAM on C128 to 64 KB and achieve higher resolutions (e.g. 80×50 text), though not widely used in terminal programs.

In short, the software stack for a Commodore terminal will usually involve a lightweight terminal emulator on the Commodore, possibly augmented by an external program or device to handle networking. The good news is **today's Commodore terminal programs are quite advanced**, supporting color, protocols, and higher speeds – making the 8-bit user experience with modern systems surprisingly comfortable. With the right setup, you can *SSH into a Linux machine, browse through system logs, send commands to your robot, or even run text-based UIs* (think `top`, `nano`, etc.) all from the Commodore's keyboard.

Security Features & Hardening Measures

Using a C64/C128 as a terminal can actually enhance security for a robotics control station. These machines are so simplistic by modern standards that they present an **extremely small attack surface** – and we can leverage hardware mods and software practices to harden them further:

- **Limited Connectivity = Fewer Vectors:** A stock Commodore has *no network stack, no background processes, no wireless interfaces*. When employed as a dedicated console (especially via direct serial), it's essentially "air-gapped" except for the explicit link to the host robot system. This isolation can protect critical systems from typical malware – for instance, the Commodore cannot run modern viruses or spyware targeting Windows/Linux. One tongue-in-cheek example is using a Commodore SX-64 as a two-factor authentication device: it's so obscure that a thief or hacker would not even know how to operate it, providing security through obscurity [57]. The same principle applies to a Commodore terminal – an attacker on the network can't easily plant a keylogger or trojan on a C64 they can't even execute code on. As long as the physical unit is secured (in a locked lab, for instance), the risk of compromise is very low. By contrast, a standard PC terminal might be attacked via remote desktop vulnerabilities or OS exploits – those simply do not exist on a C64.

- **One Fixed Application:** We can lock down the Commodore to run only the terminal program, preventing misuse. For example, burning the terminal client into a ROM cartridge ensures the machine boots straight into that program and cannot easily execute anything else. An **EPROM cartridge** or EasyFlash can hold the terminal software – with autoboot, the user is presented with a login prompt (from the host) and no normal READY prompt. Without disk access or a way to break into BASIC, an unauthorized user at the terminal can't do much except interact with the host system as intended. Some terminal programs themselves can be configured to **disable the RUN/STOP key or exit commands**, effectively *kiosk-ing* the Commodore. Moreover, the C64 has a toggle for its **RESTORE key (NMI)** which one could hardware-disable to prevent breaking out of running code. By using a read-only cart and perhaps disconnecting the IEC serial bus (to prevent someone plugging in an unknown disk drive), you create a single-purpose appliance.

- **Secure Communication:** While the Commodore can't do strong encryption alone (aside from some impressive feats – someone implemented HMAC-SHA1 for TOTP on a 1 MHz 6510 [58]), we can integrate encryption via hardware. The recommended approach is what we discussed earlier: use an intermediary (like a Raspberry Pi or WiFi modem with TLS support) to handle secure protocols. That way, the Commodore's link to the host is encrypted end-to-end. For example, running an **SSH session**: The Commodore communicates over a local serial cable to a Pi; the Pi runs the SSH client to

the robot's server, thus securing the traffic with modern ciphers. From the user's perspective, it feels like using SSH on the C64 (some have indeed done this, using Pi-based modems to connect to secure BBS systems). Another strategy is using the C64 as a true *"dumb terminal"* via an actual serial terminal server – e.g., connect the user port to a secure terminal server device that then connects to the robot's controller over a separate network. This isolates the Commodore from any direct network exposure. It's similar to how high-security systems use serial consoles (which in the Commodore's case, can't be hacked remotely without physical access).

- **Authentication & Logging:** One might incorporate additional authentication factors on the Commodore. Since the machine is fully under our control, we could require, say, a physical key or token to be present. An imaginative (but illustrative) option: use the user port to interface a keycard reader or a numeric keypad that the user must unlock before the Commodore terminal activates. The robot's host system could also expect a certain "unlock sequence" from the terminal on startup, providing mutual authentication. While the Commodore cannot handle public-key crypto well, it could store a pre-shared key or OTP generator. The mentioned SX-64 two-factor project proves even a C64 can generate 6-digit TOTP codes using an RTC and HMAC in assembly [58]. Thus, a C64 could be programmed to require a one-time passcode (displayed on its screen or entered via keyboard) that is synchronized with the host. This is overkill, but shows that *security isn't an afterthought even with retro hardware*.

- **Physical Tamper Resistance:** Commodore machines are robust and can be further fortified. One can literally add a lock to the C64's case or a switch that controls the power. For instance, wiring a key switch in series with the power button or reset line: only authorized personnel with the key can turn on or reset the terminal. The C64's case has plenty of room for such modifications (the User Port area or cassette port hole can host a small key lock). Additionally, removing any peripherals like datasette or cartridge port when not in use can prevent an attacker from inserting a malicious cartridge or tape payload. (While such attacks are extremely improbable, in theory an attacker with brief physical access could insert a cartridge that takes over the machine. If we assume a high-security environment, simply keep those ports covered or locked.)

- **Minimal Attackable Code:** The simplicity of the terminal program itself can be an asset. A well-tested C64 terminal (like those used for decades in BBS scenes) is unlikely to crash or be exploited by malformed data – it just prints characters. There is no buffer overflow in a 40-year-old PETSCII print routine that hasn't already been discovered. By contrast, a modern PC terminal might have zero-day vulnerabilities in its font rendering or escape sequence handling. That said, one should still configure the terminal carefully: e.g., **disable any disk logging** (so no data from host gets saved to disk unless needed) and **beware of escape sequences**. Some terminals (even vintage ones) can be tricked by certain control codes – for instance, XTerm had attacks where a log file with special characters could execute commands. Commodore terminals are much simpler; however, an ANSI bomb could try to switch the C128 to 40-col mode or mess with the display. As a precaution, use terminal software that is known stable and consider filtering input on the host side if extremely paranoid.

- **One-Way Data Flow (Optional):** In a highest-security scenario, the Commodore could be used as an **output-only** device (display) with input being mediated. For example, the C64 could be connected *only via its video output* to a display system and only via its keyboard (through a converter) to the robot – meaning it's functioning as a **trusted KVM** separation. This however requires significant

custom setup (there exist projects using C64 keyboards as USB keyboards via adapters, and video capture of C64 output to PC). A simpler variant: if the requirement is just to monitor the robot's status securely (read-only), one could hook the Commodore as a serial printer or status terminal that doesn't accept commands back. But typically, you'll want bidirectional control, so this is an extreme measure.

In practice, a Commodore acting as a secure terminal will rely on **external security of the host connection** (i.e., encrypted link via Pi/PC) and the inherent unreachability of the C64 from modern attack vectors. The *security mindset* is different: rather than patching software weekly, you have a device that only does one thing and does it offline. An anecdotal benefit: any thief or hacker who steals the physical unit will have a hard time even figuring out how to use it – as Hackaday quipped, *"any thief would need to spend quite some time figuring out how to operate it."* [57]. This obscurity and the inability to run unauthorized code make the C64/C128 quite secure compared to a generic IoT terminal. Of course, standard good practices still apply: guard the room it's in, secure the cables, and keep backups (since the hardware is aging).

Real-World Upgrade Examples

Finally, let's look at a few **concrete examples of Commodore hardware mods in action** that embody the above aspects, proving that C64s and C128s can serve in 2020s projects:

- **Idun Cartridge (C128/C64 + Raspberry Pi):** The Idun project (2025) we discussed is perhaps the most comprehensive example. In practice, users have assembled the Idun kit which is a through-hole board plugging into the expansion port, mounting a Raspberry Pi Zero 2 on top [13]. Once running, it provides a DOS-like menu on the Commodore, allowing selection of apps and even a built-in web interface for management [15] [17]. One mode is a **VT100 terminal directly to the Pi's Linux** – the Commodore essentially becomes a smart terminal for the Pi with a flip of a menu option [14]. People have used it to Telnet into BBSes (via the Pi's Wi-Fi/Ethernet) and even stream music from a PC to the SID chip [17] [18]. For a robotics project, Idun demonstrates how you might *launch a custom control program on the Pi from the C128's menu* and have the Commodore screen be that program's interface. Meanwhile, heavy logic (path planning, vision processing) runs on the Pi, safe from the Commodore's limitations. Security-wise, all network connectivity is through the Pi (which can run modern firewalls, encryption), and the Commodore is just the local console.

- **Leif Bloomquist's PiZero Interface (C64 User Port):** Another real build: a half-finished project described in 2018 connected a Pi Zero W via the user port [59]. It provided two modes: use any C64 terminal software to get a Linux shell on the Pi, *or* switch and display the Pi's HDMI/composite output on a monitor (essentially using the C64's monitor for Pi video) [42]. Both the C64 and Pi run in parallel, enabling creative dual-screen or dual-processor possibilities [60]. One idea was to run **TCPSer on the Pi** and let the C64 use it for full TCP/IP (including SSH) which typical Arduino-based WiFi modems couldn't handle [61]. Schematics (Fritzing files) were provided showing the wiring between Pi GPIO and C64 user port, using a level translator [40] [41]. This is a great example of **GPIO-style wiring** to marry old and new: with just a few wires and a custom PCB, the user created a hybrid system where the C64's keyboard and screen drive a Pi, and the Pi can feed video back. Although not specifically about robotics, one can imagine substituting the Pi with a microcontroller or an SBC connected to a robot – the principle is the same.

- **GGLabs GLINK & WiFi Modems (Serial Upgrades):** There are off-the-shelf serial devices for C64 that facilitate connections. The **GLINK-LT** is a modern clone of Commodore's RS-232 cartridges, plugging into the user port and supporting up to 38.4 kbps with hardware flow control. A reviewer found that with GLINK and a terminal program, a C64 could function quite well as a serial terminal, even displaying ANSI BBS menus at 2400–9600 bps comfortably [62]. **WiFi modem** devices (e.g. from RetroRewind, or DIY ESP8266 mods) deserve mention: they plug into C64 user port or USER > RS232 adapter and emulate a modem but actually connect to WiFi. In the BBS story earlier, the user bought a WiFi Modem from Retro Rewind to connect his C128 to an online BBS [63]. After adjusting settings (ensuring the modem used ANSI translation instead of PETSCII) and toggling DesTerm into "cheap RS232" mode for user port compatibility, he successfully logged into the BBS and planned to set up a local telnet server for home control [64] [65]. This shows that **with minimal hardware** (just a $30 WiFi modem), a Commodore can join modern networks and control systems via telnet. While telnet is not encrypted, one could run a VPN or tunnel on the network to secure it, or use an ESP32-based modem that might support TLS. In any case, these devices are quick solutions to get a Commodore talking to modern devices over TCP/IP without opening the computer.

- **Turbo Chameleon 64 (FPGA cart):** The Chameleon cartridge is often touted as *"the only C64 cart you will ever need"* [66]. In a retro-battlestations post, a user highlighted its impressive feature set: *VGA output, optional Ethernet via RR-Net, dual 1541 drive emulation, multiple KERNAL ROMs, turbo mode, PS/2 keyboard/mouse inputs, and full 16 MB REU* [10]. We've touched on many of these features in isolation; Chameleon combines them. For a secure terminal scenario, a Chameleon-equipped C64 could connect to an Ethernet LAN (using RR-Net Mk3) and run Contiki or Novaterm IP clients. It could output to a VGA monitor (easier to find and perhaps allowing the terminal to be on the same KVM as other modern machines). It could even be used *standalone* – interestingly, Chameleon can run **without a C64 attached** (as an FPGA emulating a C64) [67]. So one could imagine deploying a "C64 terminal" that is actually the Chameleon running headless with a VGA and PS/2 keyboard – the user experience is the same, though purists may prefer the real keyboard feel. For our purposes, this highlights that there are **integrated upgrade solutions** already proven in the field.

- **Custom GPIO Projects:** Beyond computing modules, there are examples like using the C64 user port to control robotics or automation directly. In the 1980s, Commodore published application guides for using the user port to drive stepper motors, lights, etc. Modern takes include using Firmata as mentioned (C64 controlling Arduino with sensors) [27], or an **Arduino-based C64 MIDI interface** (showing the C64 interacting with external hardware in real-time) [68]. While not security-focused, these illustrate that a C64 with some code can handle real-world I/O – one could have the Commodore monitor a few critical signals from the robot (through user port bits or joystick port) as an extra safety layer (e.g., an alarm on a certain condition, independent of the main robot controller).

- **Two-Factor Authenticator C64:** Although not an "access terminal" to robotics, the hack of turning a Commodore into a 2FA code generator is a compelling demonstration of security+retro synergy. [Cameron Kaiser] converted an SX-64 to generate TOTP codes using hand-optimized SHA-1 and an RTC [58] [69]. It underscored that even cryptographic algorithms can run on C64 in a "reasonable" time. For a robotics project requiring an **authenticator** (say to unlock command capabilities), one could similarly use the Commodore as the device that generates or checks one-time codes. This is an unconventional use, but it might appeal in a scenario where the entire robotics system is designed

around a retro aesthetic or constraint – ensuring even authentication is done on 8-bit hardware for auditability.

Wiring & Interfacing Diagrams: While we can't embed full schematics in text, consider Fig.1 below as an example of connecting a Commodore 64 to an Arduino microcontroller via the user port. The C64's TXD and RXD lines (on User Port pins B&C for Rx and M&N for GND) go through a level shifter to the Arduino's serial pins. The diagram shows a simple MAX232 circuit accomplishing this conversion, along with connections for RTS/CTS handshaking (using inverters for the CIA's control lines) [24] [25]. By following such diagrams, you can reliably link the Commodore's user port to any TTL serial device (Arduino, Raspberry Pi, etc.). Similarly, Fig.2 (if shown) might illustrate the Idun cartridge layout – where the Propeller MCU sits between the C128's expansion edge and a Raspberry Pi's GPIO – though that is a more complex assembly. The key takeaway is that **only a few wires or a small board are needed to wire up old and new**: GND to GND, TX to RX, RX to TX, plus perhaps level shifting and resets. With that, and the software/hardware described, your Commodore 64 or 128 becomes a secure, effective terminal for a cutting-edge robotics project – uniting the reliability of old-school design with the connectivity of modern tech.

Figure 1: Example C64 interface – Commodore 64 running a Firmata client to control an Arduino. The C64 screen (80-col mode via software) displays status of Arduino-connected devices (LED, relay, servo, etc.), demonstrating use of the user port for bidirectional I/O at 2400 bps [27]. Such setups can be adapted to robotics for monitoring sensors and toggling actuators securely.

Conclusion: The Commodore 64/128, enhanced with 2020s hardware mods, can indeed serve as secure access terminals. They offer *just enough* performance with accelerators and memory expansions to handle interface tasks, and their plethora of I/O (augmented by new cartridges) allows connection to modern controllers. By running well-crafted terminal software and offloading heavy computing to companion devices, a Commodore terminal provides an isolated, user-friendly control link to a robot. And beyond practicality, there's an undeniable charm (and perhaps security) in using a 40-year-old computer to oversee an advanced AI robot – a reminder that sometimes simplicity and resilience go hand in hand. [70] [15]

Sources: Modern C64/C128 hardware news [70] [71]; Community projects bridging Commodore and Pi [42] [15]; Technical details on accelerators and expansions [4] [10]; Terminal software features and user experiences [72] [49]; Security perspectives using C64 in new ways [57] [28].

---

[1] [2] [3] [8] [70] [71] TurboMaster V3: A Modern Take on the Classic Commodore 64 Accelerator - The Oasis BBS
https://theoasisbbs.com/turbomaster-v3-a-modern-take-on-the-classic-commodore-64-accelerator/

[4] [5] [6] MCL64 – World's Fastest Commodore 64 – MicroCore Labs
https://microcorelabs.wordpress.com/2021/04/19/mcl64-worlds-fastest-commodore-64/

[7] RAD Expansion Unit - The MagPi magazine - Raspberry Pi
https://magpi.raspberrypi.com/articles/rad-expansion-unit

[9] [11] [36] Turbo Chameleon 64 - C64-Wiki
https://www.c64-wiki.com/wiki/Turbo_Chameleon_64

10 12 19 67 Turbo Chameleon 64 Cartridge for Commodore 64-- VGA, Ethernet, SD cards, PS/2 KB/Mouse, more : r/retrobattlestations
https://www.reddit.com/r/retrobattlestations/comments/2ajhe8/turbo_chameleon_64_cartridge_for_commodore_64_vga/

13 14 15 16 17 18 Idun-Cartridge Connects C128 to Raspberry Pi - The Oasis BBS
https://theoasisbbs.com/idun-cartridge-connects-c128-to-raspberry-pi/

20 TurboMaster V3: Compatibility with Commodore 128 and Port ...
https://theoasisbbs.com/turbomaster-v3-compatibility-with-commodore-128-and-port-expanders/

21 Double the Z80 clock speed in your Commodore 128 - YouTube
https://www.youtube.com/watch?v=p9dZlJVGQig

22 23 Commodore C64 User Port · AllPinouts
https://allpinouts.org/pinouts/connectors/cartridges_expansions/commodore-c64-user-port/

24 25 C64 User port -> DB9 RS232 UP9600 adapter (MAX232) diagram - Commodore 64 - Lemon64 - Commodore 64
https://www.lemon64.com/forum/viewtopic.php?t=84705

26 44 45 46 47 72 GitHub - mist64/ccgmsterm: CCGMS Future, a terminal program for the Commodore 64
https://github.com/mist64/ccgmsterm

27 28 30 31 Interface Your C64 With Arduinos Through Firmata | Hackaday
https://hackaday.com/2019/04/09/interface-your-c64-with-arduinos-through-firmata/

29 Connectors Commodore 64 – My old computer
https://myoldcomputer.nl/technical-info/datasheets/connector-pinouts/connectors-commodore-64/

32 A New Commodore C128 Cartridge | Hackaday
https://hackaday.com/2023/04/17/a-new-commodore-c128-cartridge/

33 34 57 58 69 Generating Two-Factor Authentication Codes With A Commodore 64 | Hackaday
https://hackaday.com/2022/11/14/generating-two-factor-authentication-codes-with-a-commodore-64/

35 Turbo Chameleon V2 - icomp - en
https://icomp.de/shop-icomp/index.php/en/produkt-details/product/Turbo_Chameleon_64.html

37 Arduino library for serial IEC bus communication - Commodore 64
https://www.lemon64.com/forum/viewtopic.php?t=85755

38 Commodore 128 - 80 Columns w/OSSC - AtariAge Forums
https://forums.atariage.com/topic/308343-commodore-128-80-columns-wossc/

39 List of Commodore 128 commercial software
https://rclassiccomputers.com/c128software/

40 41 Interface the Raspberry Pi Zero W to Commodore 64 – OSH Park
https://blog.oshpark.com/2017/11/24/interface-the-raspberry-pi-zero-w-to-commodore-64/

42 43 59 60 61 Raspberry Pi Zero W / Commodore 64 Interface Board - Commodore 64 - Lemon64 - Commodore 64
https://www.lemon64.com/forum/viewtopic.php?t=68029

48 C128 80 Column software - Commodore 64 - Lemon64
https://www.lemon64.com/forum/viewtopic.php?t=38159

49 50 54 55 63 64 65 obsolete.computer - Accessing a BBS from a Commodore 128
https://obsolete.computer/geekery/accessing-a-bbs-from-a-commodore-128.html

51 52 The many Operating Systems of the Commodore 64
https://lunduke.substack.com/p/the-many-operating-systems-of-the

53 Does the C64 have TCP/IP capabilities? - Hacker News
https://news.ycombinator.com/item?id=13063767

56 80 column mode terminal program with Petscii graphics? - Reddit
https://www.reddit.com/r/c128/comments/tziklp/80_column_mode_terminal_program_with_petscii/

62 C64 on trial: Is it any good as a serial terminal? - YouTube
https://www.youtube.com/watch?v=8YwekFqma6Q

66 The only C64 cart you will ever need?! - Turbo Chameleon 64 V2
https://www.youtube.com/watch?v=04qBsf6et2s

68 Commodore 64 Arduino MIDI interface : r/c64 - Reddit
https://www.reddit.com/r/c64/comments/1gcpmwm/commodore_64_arduino_midi_interface/